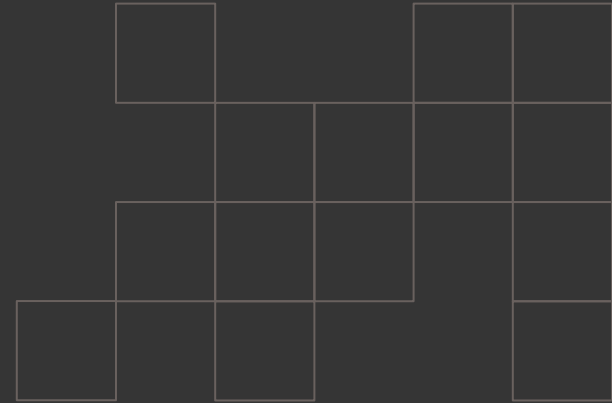


Nigel Black

ADV CS III Capstone Project: F1 Elo Engine



What is an ELO rating?

ELO tells us how good you are, specifically how likely it is you will beat someone else

- If you lose to someone with a higher ELO/iRating than you, you lose less.
 - If you lose iRating to someone with a lower rating than you, you lose more.
- If you beat someone, you gain more iRating the higher of a score they have compared to you, and less if they have a lower score



iRacing[®]



Project Overview



Let me break it down for you Mark

- ELO systems allow us not only a quantifiable, empirical comparison of drivers in a sport where skill is often very hard to quantify, but also a gives us a rough measure of who would beat who in equal machinery
- We also get a mathematical answer to who are the GOATs of the sport, although it is an imperfect and subjectively designed system

How will the system work?

- Pulls live race result data from the Jolpi F1 API across every season 1950–present (nlohmann/json)
- Computes Elo power ratings for every driver and constructor in F1 history
- Outputs ranked leaderboards: all-time GOATs, active driver rankings, constructor standings

So how do we apply this to F1?

Part 1

We use a scoring system similar to iRacing, where drivers earn rating for their teams based on where they place in the team.

Part 2

We use those scores to add/subtract rating from each team using a baseline of 2000 ELO, and using the ranking that this produces, scores how well a driver placed in relation to the team's expected finish. (ex. The second best team is expected to place 3rd and 4th. If one of their drivers wins, he will see a bigger boost as he overperformed).

Part 3

We combine this with a separate metric which looks at how far above or below said driver finished above his teammate, adding or subtracting driver ELO accordingly, starting at 1500 ELO.

Additional Gains

Drivers then get additional gains for placing 1st, 2nd or 3rd, winning championships, consecutive or multiple career championships, etc.



Implementation!

This is all found in my class called RatingEngine.cpp



```
std::unordered_map<std::string, double> driverMovement; // driverId -> net movement this race
```

```
for (int i = 0; i < n; ++i) {
    Driver* currentDriver = grid[i];
    int actualPos = i;
    int expectedPos = expectedPosMap[currentDriver->id];

    // --- Career Weight (arctan curve) ---
    // Rookies start at 0.1, asymptotically approaching 1.0 around 120 races.
    // Prevents a single fluke result from permanently inflating a new driver.
    double careerWeight = std::atan(currentDriver->raceCount / 40.0) / (M_PI / 2.2);
    if (careerWeight < 0.1) careerWeight = 0.1;

    // --- Tanh Smoothing ---
    // Compresses extreme position swings (winning from P18 due to a safety car, etc).
    // Positive rawDelta = outperformed expectations (expected lower, finished higher).
    double rawDelta = (double)expectedPos - (double)actualPos;
    double smoothedDelta = std::tanh(rawDelta / 15.0) * 15.0;

    double movement = (smoothedDelta * (kFactor * 0.6)) * careerWeight;
}
```

```
// Sort a copy by potential descending - this is the model's predicted finishing order.
std::vector<Competitor> expectedGrid = contenders;
std::sort(first: expectedGrid.begin(), last: expectedGrid.end(), comp: [](const Competitor& a, const Competitor& b) ->bool {
    return a.potential > b.potential;
});

// Build a lookup: driverId -> expected position index (0 = predicted winner)
std::unordered_map<std::string, int> expectedPosMap;
for (int j = 0; j < (int)expectedGrid.size(); ++j) {
    expectedPosMap[expectedGrid[j].d->id] = j;
}
```

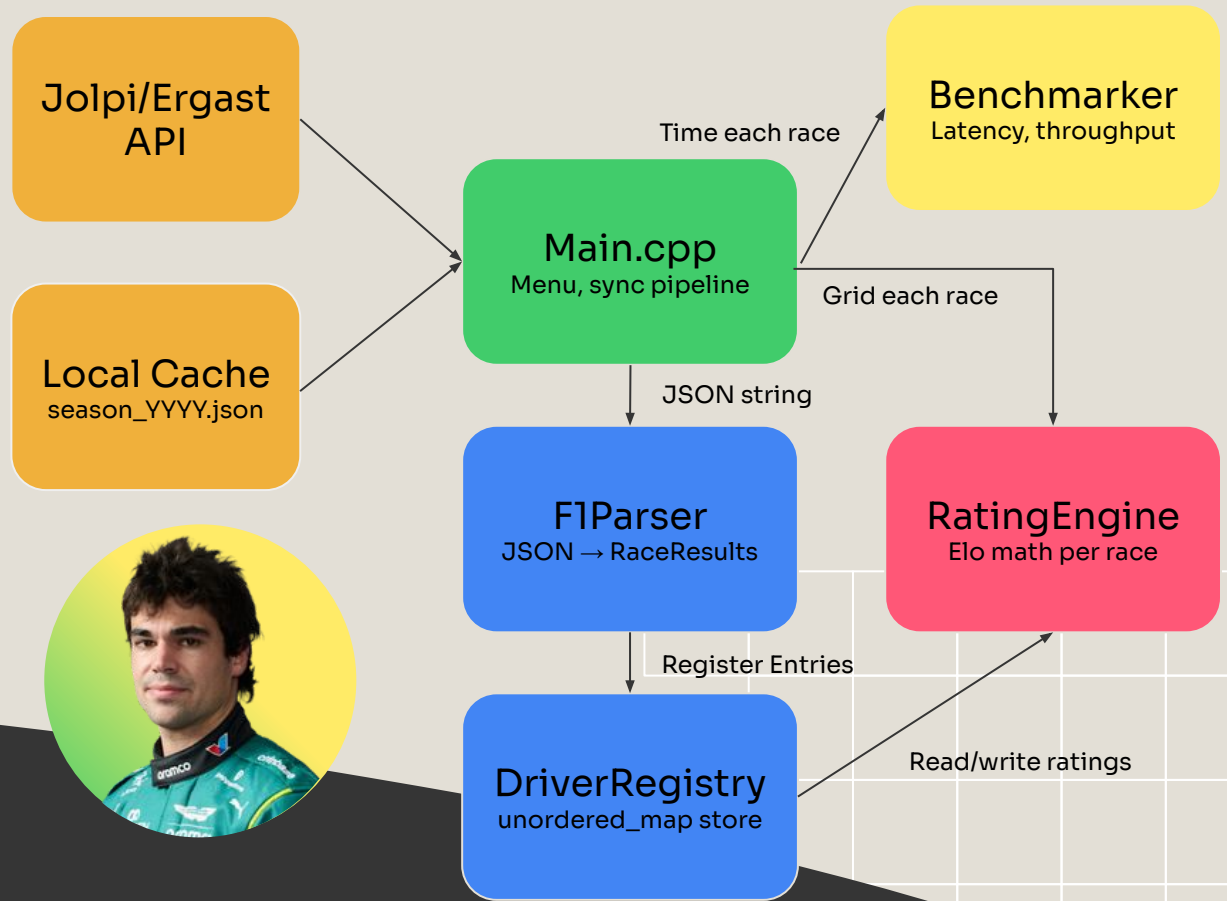
```
// --- Teammate Duel (standard head-to-head Elo) ---
// Find this driver's first teammate in the grid and run a clean
// head-to-head exchange. Controls for car quality – both drivers
// have the same machinery, so relative finishing is a pure skill signal.
for (int j = 0; j < n; ++j) {
    Driver* peer = grid[j];
    if (peer->teamId == currentDriver->teamId && peer->id != currentDriver->id) {
        double winProb = 1.0 / (1.0 + std::pow(10.0,
            (peer->allTimeRating - currentDriver->allTimeRating) / 400.0));
        double actualOutcome = (actualPos < j) ? 1.0 : 0.0;

        double duelShift = (kFactor * (actualOutcome - winProb)) * careerWeight;
        currentDriver->allTimeRating += duelShift;

        // Absorb duel shift into the tracked movement so it feeds the team calc
        driverMovement[currentDriver->id] += duelShift;
        break;
    }
}
```



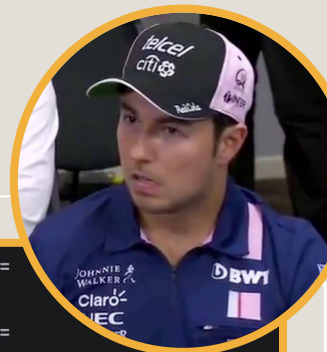
System Architecture



Data Structure Selection

F1 has had a lot of drivers, and a lot of races.

- We have to hold 1,100+ drivers in a structure which gets looked up 22,000 times during a sync.
- `std::unordered_map` uses a hash function on the driver ID giving $O(1)$ average lookup.
- `std::map` gives $O(\log n)$



```
=====
DATA STRUCTURE COMPARISON: unordered_map vs std::map
=====
```

```
Setup: 1100 drivers, 22000 lookups, averaged over 5 trials
```

```
std::unordered_map  avg time : 5.2125 ms   (O(1) avg)
std::map             avg time : 27.8415 ms  (O(log n))
Speedup              : 5.34x faster
```

```
Theoretical O(log n) factor at n=1100: ~10.1 comparisons per lookup vs ~1 for hash
```


Performance Results

F1 RATING ENGINE — PERFORMANCE BENCHMARK REPORT

THROUGHPUT

Total races processed : 154
Total drivers processed : 2310
Total sync wall time : 64878.697 ms (64.879 s)
Races per second : 2.4

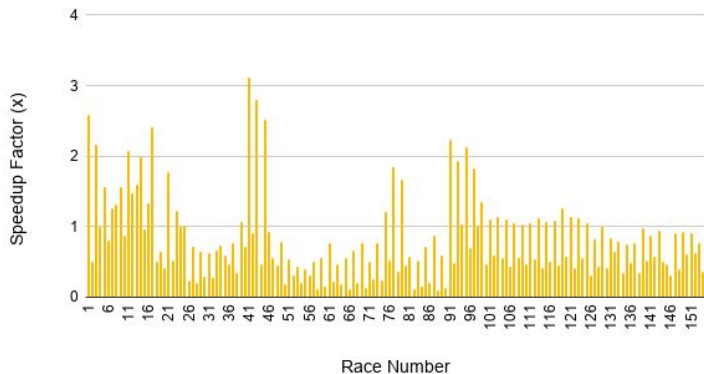
LATENCY (per race session)

Average : 0.3128 ms
Minimum : 0.0182 ms
Maximum : 1.2768 ms

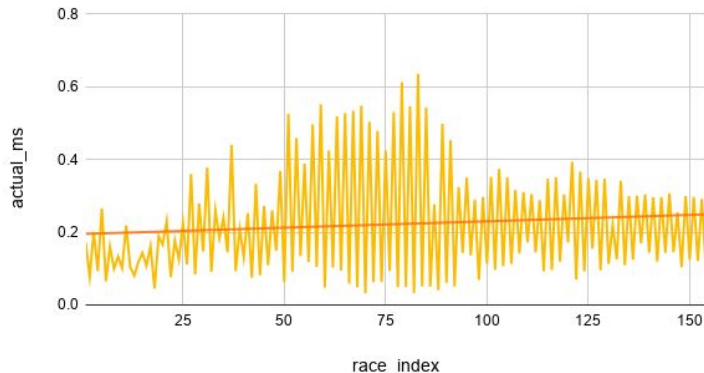
COMPARATIVE ANALYSIS (vs. naive $O(n^2)$ all-pairs baseline)

Our avg latency : 0.3128 ms
Naive avg latency (est) : 0.4229 ms
Speedup factor : 1.35x faster

Speedup Factor per Race

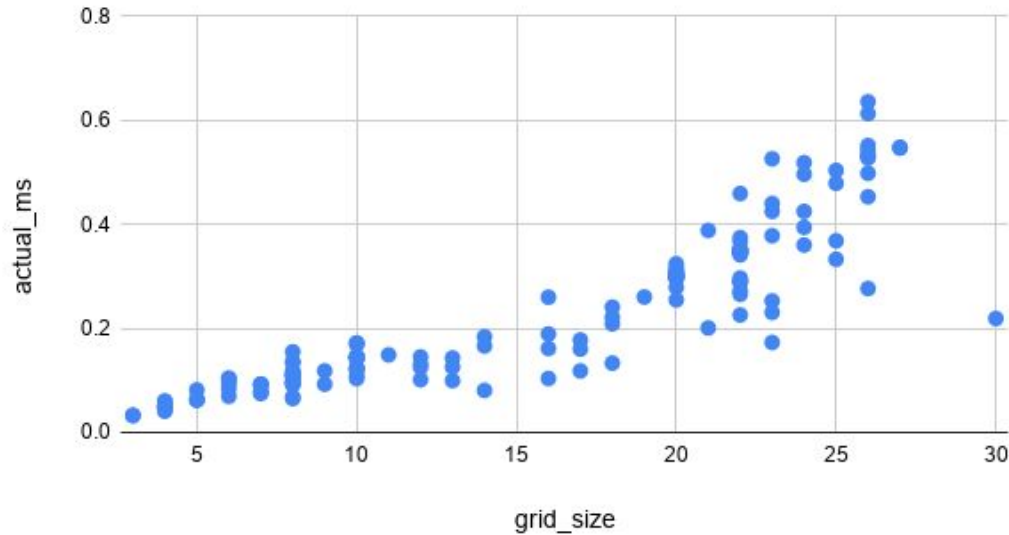


Latency Over Time



Complexity Verification

actual_ms vs. grid_size



Challenges Faced



Extremely Interesting Words

Challenge 1

Having to make sure the season that's in progress doesn't cause issues. Scaling factor for shorter seasons would be applied + crowning WDC after 4 races.

Challenge 2

Perfecting the math. This system heavily favors experienced and gives lots of drivers preferential treatment. Piastri ranked extremely low.

Challenge 3

Scaling season sizes so drivers from 40+ years ago are given *reasonably* fair representation in spite of shorter seasons/careers because they kept dying.

Conclusion

What did we find?

Arigato!

Averaged sub-millisecond latency across 75 years of data. `unordered_map` proved 3-5x faster than `map` for this access pattern.

What can we do in the future?

DNF weighting, tweaks and improvements to the calculations. Queries on historical matchups, driver/manufacture stat lookup

